

Deployment, observability and capstone

AI Engineering: GenAI and Agentic Systems

What you build today: the whole system on your cloud, and a number for the cost per resolved ticket.

Two tiers, one set of images

Tier	Who	Proves
Session path	you, today	serverless containers, least privilege, readiness, metrics, alarms, budget, rollback
Reference stack	instructor, live	managed agent runtime, the escalation gate enforced by the platform, native guardrails, a managed vector index

Nothing on the Session path is thrown away.

Containers that behave

- Two stages, non-root, dependencies locked
- Weights and indexes baked at build, loaded once at startup
- Readiness on `/readyz`, liveness on `/healthz`
- Graceful shutdown so in-flight requests finish
- No credentials in any layer. Inspect the layers to prove it

Least privilege, by name

- AWS: one execution role per function; only policy and agent may `bedrock:InvokeModel`, on three model ARNs, never `foundation-model/*`
- GCP: one service account per service; only policy and agent hold `roles/aiplatform.user`; nothing holds Editor
- The synth test refuses a wildcard before anything deploys

Where the approval gate lives now

- In the loop: it refuses to execute `escalate` without approval
- AWS Reference stack: the gateway's Cedar policy denies `escalate` unless the caller is the approvers role; shown from the CLI with a signed MCP request
- GCP Reference stack: the MCP service is private to the agent identity and Model Armor screens the prompt first
- Same rule, two places, different failure modes. That is defence in depth

Native guardrails

- AWS: Bedrock Guardrails, prompt attack at low strength, PII anonymised, grounding checked
- GCP: Model Armor template, injection and jailbreak at medium confidence, sensitive data, malicious URLs
- Low strength is deliberate: high blocks about 3 percent of ordinary support prompts
- Watch the intervention metric before tightening

Observability that is already there

Signal	AWS	GCP
Traces	X-Ray, CloudWatch Transaction Search	Cloud Trace
Metrics	Lambda, Bedrock per model, your counters	Cloud Run, your counters
Dashboard	CloudWatch, generated	Cloud Monitoring, generated
Alarms	5xx, p95 latency, to SNS	5xx, p95 latency, to email
Budget	50, 80, 100 percent	50, 80, 100 percent

Silent degradation, made loud

- A model that answers slower: p95 latency
- A retriever that returns nothing: refusal rate
- A judge that drifts: faithfulness on the harness
- A loop that wanders: steps and cost per resolution

Those are the panels. Everything else is decoration.

Serverless, and what it costs to be idle

	AWS Lambda	GCP Cloud Run
Idle	zero; images and one secret, under 1 USD a month	zero with min instances 0
Cold start	20 to 40 seconds for the agent image; provisioned concurrency buys it back at about 2.30 USD a day	20 to 40 seconds for the agent image
Stop	reserved concurrency 0	scale to zero
Destroy	zero	zero

App Runner is closed to new customers; ECS Express Mode is the always-warm option.

Measure both cold and warm p95 yourself.

Rollback you have run

- A rollback that has never been executed is a plan, not a rollback
- Deploy a configuration that cannot become ready on purpose, watch readiness fail, roll back, watch it recover
- Write down the command. It goes in the runbook

Capstone: cost per resolved ticket

1. Twenty tickets through the agent loop, four at a time: cost per resolution
2. The router you wrote in front: confident P0 to escalation with no model call, P3 to the Economy model, the rest to the Workhorse
3. Same twenty tickets: cost per resolution again
4. Argue with the number before the architecture

Acceptance criteria

1. Public HTTPS endpoint answering a real ticket end to end
2. No credentials in the image, verified by inspecting layers
3. Cold-start and warm p95 latency both measured and written down
4. A dashboard whose request and latency panels move with your tickets (token spend per model on AWS), and a budget alarm that exists
5. A rollback command that you have actually run once

make `session06` reports every test passed.

Debrief

- Which failure this course would have reached production in your setup?
- What is your cost per resolved ticket, and what would Northwind accept?
- Which component do you replace first when the vendor changes?
- What did the framework port hide, and does it matter at 3am?